

SIMATS ENGINEERING

ASSESSMENT TOOL 2

**COURSE CODE: CSA14
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:100

Annexure A

CASE STUDY

Code of Conduct:

*I am M.Niharika(192365019)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:Niharika

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

CASE STUDY

Set 1

Q1. Both parse trees and syntax trees are used to represent program structure, but differ significantly in complexity and abstraction level. Compare these representations in terms of memory usage, ease of traversal, suitability for optimization, and their role in different phases of compilation such as semantic analysis and code generation. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, representation choice affects compiler performance. Finally, selecting the right abstraction improves overall efficiency.

Compare Parse Trees and Syntax Trees

A **parse tree** and a **syntax tree (Abstract Syntax Tree or AST)** are both used to represent the structure of a program, but they differ in their level of detail and purpose.

1. Memory Usage

- A **parse tree** contains all grammar symbols, including terminals and non-terminals. Therefore, it requires **more memory**
- A **syntax tree** removes unnecessary grammar details and keeps only important program constructs, so it requires **less memory**.

2. Ease of Traversal

- Parse trees contain many nodes, making traversal **more complex and slower**.
- Syntax trees are smaller and simpler, making them **easier and faster to traverse**

3. Suitability for Optimization

- Parse trees are not ideal for optimization because they contain many grammar-specific details.
- Syntax trees are highly suitable for optimization because they directly represent expressions, statements, operators, and operands.

For example, for:

$a + b * c$

the syntax tree clearly shows that $b * c$ must be evaluated before $a +$.

4. Role in Compilation Phases

1. **Parse Tree:** Mainly used during the **syntax analysis/parsing phase** to verify whether the program follows the grammar.
2. **Syntax Tree:** Mainly used during **semantic analysis, optimization, and code generation**.
3. During semantic analysis, the syntax tree helps perform **type checking, scope checking, and symbol resolution**.
4. During code generation, the syntax tree provides a simpler structure from which intermediate or machine code can be generated.

Conclusion

A parse tree provides a **complete and detailed grammatical representation**, while a syntax tree provides a **compact and meaningful representation** of the program. Therefore, syntax trees generally improve memory efficiency, traversal speed, optimization, and code generation.

Q2. A compiler designer must choose between bottom-up and top-down evaluation strategies for implementing syntax-directed definitions. Justify this choice by analyzing grammar restrictions, attribute dependencies, parsing efficiency, and the ease of integrating semantic actions within each approach. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, evaluation strategy influences implementation complexity. Finally, choosing the right method ensures better performance.

Bottom-Up vs Top-Down Evaluation of Syntax-Directed Definitions

A compiler designer can use **top-down** or **bottom-up evaluation** to implement syntax-directed definitions. The choice depends on the grammar, attribute dependencies, parsing method, and ease of implementing semantic actions.

1. Grammar Restrictions

- **Top-down evaluation** is suitable for grammars used with LL or recursive-descent parsers.
- Such grammars may require removing **left recursion** and applying left factoring.
- **Bottom-up evaluation** works with a wider range of grammars, including **left-recursive grammars**.

2. Attribute Dependencies

- Top-down evaluation is suitable for handling **inherited attributes**, where information is passed from parent to child.
- Bottom-up evaluation naturally handles **synthesized attributes**, where information is calculated from child nodes and passed to the parent.
- An **attribute dependency graph** should be checked to ensure that attributes are evaluated in the correct order and that there are no circular dependencies.

3. Parsing Efficiency

- Top-down parsing can be simple and efficient for suitable grammars.
- Bottom-up parsers such as **LR parsers** can handle more complex grammars and are commonly used in compiler construction.
- Bottom-up parsing can therefore provide greater grammar-handling power.

4. Integration of Semantic Actions

- In top-down parsing, semantic actions can be performed while productions are being expanded.
- In bottom-up parsing, semantic actions are commonly executed when a production is **reduced**.
- This makes synthesized attribute evaluation particularly convenient in bottom-up parsing.

Conclusion

Top-down evaluation is preferable for simple predictive grammars and inherited attributes, while **bottom-up evaluation** is generally better for complex grammars and synthesized attributes. The compiler designer should consider grammar restrictions, attribute dependencies, parsing efficiency, and implementation complexity before selecting the appropriate strategy.

Q3. In complex arithmetic and logical expressions involving multiple operators, precedence rules, and associativity constraints, attribute dependencies may create conflicts in evaluation order. Analyze how these dependencies affect semantic rule execution and propose techniques such as dependency graphs or evaluation scheduling to resolve them. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, resolving conflicts is critical for correctness. Finally, structured evaluation improves reliability.

Attribute Dependencies in Complex Expressions

In complex arithmetic and logical expressions, different operators have different **precedence and associativity rules**. These rules create dependencies between attributes because some operations must be evaluated before others. For example, in the expression $a + b * c$, multiplication must be evaluated before addition because multiplication has higher precedence. Similarly, in $a - b - c$, the left-associative rule requires the expression to be evaluated as $(a - b) - c$.

Attribute dependencies determine the order in which semantic rules are executed. If a semantic rule is executed before the attributes it depends on are available, it can produce an incorrect result. Therefore, the compiler must identify the dependencies between attributes and determine a valid evaluation order.

An **attribute dependency graph** can be used to represent these relationships. Each attribute is represented as a node, and an edge indicates that one attribute depends on another. The compiler can then use **topological ordering** to determine the correct sequence of evaluation when the dependency graph has no cycles. An evaluation scheduler can also execute each semantic rule only after all its required attributes have been calculated.

Thus, attribute dependencies are important for maintaining the correct order of semantic rule execution. Using dependency graphs, evaluation scheduling, and proper precedence and associativity rules helps the compiler resolve conflicts and produce correct and reliable results.

Q4. A statically typed programming language enforces type checking during compilation to ensure correctness. Evaluate how such a type system helps detect errors early, prevents invalid operations, and improves software reliability, while also discussing any limitations in flexibility compared to dynamically typed systems. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, strong typing enhances safety. Finally, balancing flexibility and strictness is important.

Static Type System

A **statically typed programming language** checks the types of variables, expressions, and operations during compilation. This helps the compiler identify many errors before the program is executed. For example, if an integer variable is assigned a string value, the compiler can detect the type mismatch during compilation.

Static typing prevents invalid operations by ensuring that values are used only with compatible types. It can detect errors such as assigning incompatible values, passing incorrect argument types to functions, or performing unsupported operations on data. This improves software reliability because many common programming errors are identified at an early stage.

Static type systems also help the compiler generate efficient code because the types of values are known during compilation. They improve code readability, maintainability, and provide better support for development tools such as code completion and error detection.

However, static typing can be less flexible than dynamic typing because programmers may need to specify types explicitly and follow stricter type rules. Dynamic languages provide greater flexibility because types are checked during program execution, but some errors may only be discovered at runtime.

Therefore, a static type system improves **safety, early error detection, reliability, and performance**, but it may reduce flexibility compared with dynamically typed systems. A good programming language should balance type safety with ease of development and flexibility.

Q5. A modern programming language supports advanced features such as polymorphism, function overloading, and type inference. Analyze the challenges faced by the compiler's type system in resolving types accurately, managing ambiguity, and maintaining efficient compilation performance. These require careful consideration of attribute dependency graphs to ensure correct evaluation order without conflicts. Furthermore, advanced features increase complexity. Finally, efficient resolution mechanisms are essential.

Challenges in Advanced Type Systems

Modern programming languages support advanced features such as **polymorphism, function overloading, and type inference**, which provide flexibility and reduce the amount of code programmers need to write. However, these features make the compiler's type system more complex.

In **polymorphism**, the same function or operation can work with different types. The compiler must determine the correct type for each use. In **function overloading**, several functions can have the same name but different parameter types, so the compiler must identify which function should be called based on the given arguments. This can create ambiguity when more than one function appears to be suitable.

Type inference allows programmers to write code without explicitly specifying every type. The compiler must analyze expressions and determine the correct types automatically. This requires the compiler to track relationships between different types and solve type constraints.

Another challenge is maintaining **efficient compilation performance**. Complex type checking, overload resolution, and type inference can require significant computation, especially in large programs. The compiler must also provide clear error messages when it cannot determine a valid type.

Attribute dependency graphs can help the compiler identify relationships between type attributes and determine the correct evaluation order. Efficient techniques such as type unification, constraint solving, and caching can also improve type resolution.

Therefore, advanced type-system features improve flexibility and programmer productivity, but they increase compiler complexity. The compiler must accurately resolve types, handle ambiguity, detect errors, and maintain efficient compilation performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand